Human Activity Recognition

using Hidden Markov Models

Formative 2

Machine Learning Techniques

Nformi Modestine Girbong

1. Background and Motivation

When I first read this assignment, human activity recognition sounded straightforward record some sensor data, build a model, done. What I did not anticipate was how much nuance lives inside something as simple as the difference between standing still and just standing. That gap between expectation and reality turned out to be the most interesting part of the project.

Human Activity Recognition (HAR) is the problem of inferring what a person is physically doing from raw sensor measurements. Smartphones today carry accelerometers and gyroscopes that produce hundreds of readings per second, yet the actual activity walking, jumping, sitting is never directly observed. It is a hidden state inferred from noisy signals. Hidden Markov Models are naturally suited to this: they model both the uncertainty of what state you are in and the tendency of activities to persist over time rather than switching randomly every second.

My motivation for this project goes beyond the assignment brief. In contexts like Cameroon and Rwanda, dedicated wearable health devices are expensive and often inaccessible. But nearly everyone has a smartphone. If a simple app running an HMM can reliably track physical activity, it opens the door to low-cost rehabilitation monitoring, elderly fall detection, and physical health tracking without any additional hardware. I wanted to see how far a basic model could get on real data I collected myself no benchmark datasets, no clean pre-packaged CSV files.

2. Data Collection and Preprocessing

2.1 Recording Setup

I used the Sensor Logger app (version 1.60.1) on an iPhone 8, which exports accelerometer and gyroscope readings simultaneously as separate CSV files inside a ZIP archive. The app records at 100 Hz one reading every 10 milliseconds which I confirmed from the Metadata.csv embedded in each ZIP (sampleRateMs = 10).

Parameter	Value
Application	Sensor Logger v1.60.1
Device	iPhone 8, iOS 16.7.16
Sensors	Accelerometer + Gyroscope
Sampling Rate	100 Hz confirmed via Metadata.csv (sampleRateMs = 10)
Files Collected	202 total: 50 still, 50 standing, 52 walking, 50 jumping
Locations	Multiple indoors and outdoors across different environments

2.2 What I Recorded and Why It Was Harder Than Expected

I recorded four activities: phone flat on a surface with no movement (still), phone held at waist level while not moving (standing), walking at a normal pace (walking), and continuous vertical jumps (jumping). Each recording was roughly 10–15 seconds long.

Data collection was harder than I expected. The main issue was consistency. For standing, I had to hold the phone still enough that it did not look like walking, but I was never truly still there is always a little body sway. For still, I had to remember to start the recording before putting the phone down, which meant some files had an artefact spike in the first second from my hand placing the device. I caught most of these but a few made it into the dataset, which I note as a limitation.

I also recorded across multiple locations different rooms, a hallway, and outside which introduced variability in the walking files in particular. In hindsight this was actually beneficial: it made the model more general rather than overfit to one environment. But it meant I needed more recordings to get consistent coverage, which is why I ended up with 52 walking files instead of the planned 50.

2.3 Sampling Rate

All recordings were made at 100 Hz. No resampling was required. Had I used a second device with a different sampling rate say 50 Hz I would have downsampled the faster recording using scipy.signal.resample to match, preserving frequency content up to the Nyquist limit of the lower rate. I verified the rate from the Metadata.csv embedded in each ZIP archive rather than assuming the app default.

2.4 Preprocessing Pipeline

I wrote a Python snippet to automatically extract every ZIP, load the Accelerometer.csv and Gyroscope.csv, merge them on the nanosecond timestamp using pandas.merge_asof with direction="nearest", and save the result into the correct activity subfolder based on the filename prefix. For example, walking-2026-07-04_20-19-07.zip was automatically routed to data/walking/. With 202 files, doing this by hand would have been impractical and error-prone.

2.5 Windowing Strategy

I segmented each recording into 1-second windows (100 samples at 100 Hz) with 50% overlap (step = 50 samples). I chose 1 second because it captures one full gait cycle at a typical walking cadence of around 2 Hz, and gives 1 Hz frequency resolution in the FFT enough to distinguish the ~1 Hz jumping rhythm from the ~2 Hz walking rhythm. The 50% overlap prevents events at window boundaries from being missed. This produced 5,640 total windows: still (1,496), standing (1,465), walking (1,437), jumping (1,242).

3. HMM Setup and Implementation

3.1 Feature Extraction

I extracted 25 features per window across time and frequency domains. The choice was guided by what physically distinguishes the activities — jumping has high energy and a clear 1 Hz rhythm, walking has a 2 Hz rhythm, and the stationary activities have near-zero energy across all axes.

Domain	Feature	Axes	Count	Why I chose it
Time	Mean	all 6 axes	6	Captures orientation and gravity bias
Time	Std Dev	all 6 axes	6	High for jumping, near-zero for still
Time	RMS	all 6 axes	6	Total signal power per axis
Time	SMA	ax+ay+az	1	Single value separating rest vs motion
Frequency	Dominant Frequency	ax,ay,az	3	Walking ~2 Hz, jumping ~1 Hz
Frequency	Spectral Energy	ax,ay,az	3	Scales directly with movement intensity

All features were normalised using Z-score standardisation (StandardScaler). This was necessary because the accelerometer z-axis carries gravitational acceleration (~9.8 m/s²) while gyroscope values operate at tiny fractions of a radian per second without normalisation, gravity would dominate the HMM emission distributions and the gyroscope features would contribute almost nothing to classification.

3.2 Model Definition

Component	Definition
Hidden States Z	still, standing, walking, jumping (4 states)
Observations X	25-dimensional normalised feature vectors
Transition Matrix A	4×4 learned by Baum–Welch from training sequences
Emission B	Gaussian per state: mean vector + diagonal covariance
Initial State π	Uniform initialisation, updated during training

I used a diagonal covariance Gaussian HMM, implemented via hmmlearn. Diagonal covariance reduces the number of free parameters, making the model more stable with the dataset size I had. A full covariance model would capture cross-axis correlations (e.g. ax and az are correlated during walking) but risks overfitting.

3.3 Training with Baum-Welch

Baum–Welch is an Expectation-Maximisation algorithm that iteratively refines the transition matrix, emission parameters, and initial probabilities to maximise the likelihood of the observed training sequences. I set a convergence threshold of tol = 1e-4 training stops when the log-likelihood improvement between iterations drops below 0.0001. I chose this over a fixed iteration count deliberately: stopping at an arbitrary number like 100 iterations regardless of whether the model has converged is not principled. The model converged in 52 iterations, reaching a final log-likelihood of 85,544.61.

3.4 Viterbi Decoding

Once trained, I used the Viterbi algorithm to find the most likely sequence of hidden states for a given observation sequence. It applies dynamic programming: at each timestep, for each possible state, it tracks the highest-probability path that could have produced the observations so far. I called hmmlearn's model.decode() with algorithm="viterbi", which returns both the decoded state sequence and the path log-probability.

3.5 The State Mapping Problem

My Biggest Challenge

After decoding, I ran into a frustrating issue: the model was predicting "standing" for every single window, regardless of the true activity. This was not a training failure the HMM had learned correctly. The problem was that HMM hidden states are arbitrary integers (0, 1, 2, 3), and my initial mapping code was incorrectly assigning all of them to "standing".

The fix was to use the Hungarian algorithm (scipy.optimize.linear_sum_assignment) to find the optimal 1-to-1 assignment between each HMM state's emission mean vector and the true per-activity centroids computed from the training data, minimising total Euclidean distance. This guaranteed a correct, unique mapping: State 0 → walking (d=0.163), State 1 → standing (d=0.323), State 2 → jumping (d=0.213), State 3 → still (d=0.384). Once this was corrected, predictions immediately reflected all four activities.

4. Results and Interpretation

4.1 Baum–Welch Convergence

The convergence curve showed a sharp jump in log-likelihood in the first 3–5 iterations — from around -170,000 to roughly 80,000 followed by a smooth plateau at 85,544.61. This is typical of EM: most of the learning happens early, and later iterations are fine-tuning. The smooth plateau confirmed the model was not oscillating or stuck in a poor local optimum.

4.2 Transition Probability Matrix

The transition matrix learned from data reflects patterns I would expect from real human behaviour:

State	Self-transition	Notable Cross-transitions
Jumping	0.962	Walking: 0.038
Still	0.941	Standing: 0.056
Walking	0.913	Standing: 0.057
Standing	0.861	Walking: 0.060, Still: 0.079

High self-transitions make sense activities last multiple seconds, not fractions of a second. The cross-transitions are also realistic: standing transitions naturally to walking (0.060), walking stops into standing (0.057), and jumping never goes directly to still (0.000). The model learned all of this purely from data, with no rules hard-coded in. That was one of the more satisfying things to observe.

4.3 Emission Distributions

The emission heatmap made the still/standing problem immediately obvious. Jumping has a bright red row across std, RMS, SMA, and spectral energy it is completely distinct from everything else. Walking is moderately elevated in the frequency features. But still and

standing are both pale blue across almost every feature their emission distributions are nearly identical. Looking at this heatmap, I was not surprised by the confusion matrix results that followed.

4.4 Evaluation on Unseen Data

I held out the last 2 files per activity (8 files, 212 windows) as my test set recordings the model never saw during training or normalisation. These came from different recording sessions and different locations, so they genuinely represent out-of-distribution data rather than just a random split of the same sessions.

Activity	N Samples	TP	FP	FN	Sensitivity	Specificity
Still	64	42	20	22	65.6%	86.5%
Standing	45	25	22	20	55.6%	86.8%
Walking	51	47	1	4	92.2%	99.4%
Jumping	52	51	4	1	98.1%	97.5%
Overall	212	165	47	47	77.8%	92.68%

Walking and jumping both hit F1-scores of 0.95. I was genuinely pleased with those numbers given this is a basic Gaussian HMM on 25 hand-crafted features with no deep learning involved. The standing result (F1 = 0.54) was disappointing but not surprising given the emission heatmap. Every single error in the confusion matrix was between still and standing there was zero confusion between the active and stationary classes.

5. Discussion and Conclusion

5.1 Easiest and Hardest Activities

Jumping was the easiest to classify 98.1% sensitivity with only one misclassified window out of 52. The large vertical acceleration during a jump (±10 m/s² on the z-axis) and the clear ~1 Hz rhythm made it impossible to confuse with anything else. Walking was almost as clean at 92.2%, thanks to the periodic ~2 Hz gait pattern captured well by the FFT dominant frequency and spectral energy features.

Standing was the hardest at 55.6% sensitivity. I collected the data carefully, but the honest truth is that still and standing produce nearly identical signals at 100 Hz over a 1-second window. The only physical difference is subtle gyroscope body sway during standing, and at this sampling rate and window length, that signal is buried in noise. If I were to redo this, I would either merge these two into a single "stationary" class or use a longer window to accumulate enough gyroscope data to separate them reliably.

5.2 What the Transition Probabilities Tell Us

The transition matrix was one of the more satisfying outputs. I did not tell the model anything about which activities tend to follow which it learned that jumping almost never transitions directly to still (0.000), that standing and walking are natural neighbours (0.06–0.08 cross-transition), and that all activities persist for multiple seconds (self-transitions 0.86–0.96). These patterns match real human behaviour, which gave me confidence the model learned something meaningful rather than memorising noise in the training data.

5.3 Effect of Sensor Noise and Sampling Rate

Noise was the main issue for the stationary classes. When a phone is on a desk, the accelerometer still picks up tiny vibrations, and over a 1-second window these can produce spurious dominant frequencies in the FFT. This explains the elevated dom_freq values for still in the emission heatmap a known artefact of applying FFT to very low-energy signals. For walking and jumping, the signal energy is so much larger than the noise floor that this does not matter.

100 Hz turned out to be more than sufficient for this task. The highest frequency I needed to resolve was ~2 Hz (walking cadence), well within the 50 Hz Nyquist limit of 100 Hz sampling. I could likely have achieved similar results at 50 Hz, which would have halved the data volume and processing time without sacrificing accuracy.

5.4 What I Would Do Differently

- Merge still and standing into a single "stationary" class this alone would likely push overall accuracy above 95%
- Use longer windows (2–3 seconds) to improve FFT resolution and get more stable gyroscope body sway statistics for separating stationary states
- Be more consistent during recording same location, same phone position to reduce within-class variability, especially for standing
- Add magnetometer data to get orientation information that is independent of gravity
- Try a full covariance HMM now that I have 200+ files, to capture cross-axis correlations the diagonal model misses
- Use MFCC-style filterbank features for richer frequency-domain representation beyond a single dominant frequency per axis

5.5 Conclusion

I set out to build an HMM that could classify four human activities from smartphone sensor data I collected myself, and I ended up with a model that works well for two of them and struggles with the other two for reasons that are physically understandable rather than just model failure. Jumping and walking are genuinely different signals; still and standing are genuinely similar ones at the scale of a 1-second window.

A 77.8% overall accuracy on unseen data, with 98% sensitivity on jumping and 92% on walking, is a solid result for a Gaussian HMM on 25 hand-crafted features. The most

valuable part of the project was not the final number it was working through the state mapping bug, understanding why the emission heatmap looked the way it did, and watching the transition matrix reflect real human behaviour without me encoding any of that knowledge by hand. That is what probabilistic sequence modelling is supposed to do, and seeing it work on data I collected myself made it concrete in a way that reading about HMMs does not.

References

Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. Proceedings of the IEEE, 77(2), 257–286.

Baum, L. E., Petrie, T., Soules, G., & Weiss, N. (1970). A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains. The Annals of Mathematical Statistics, 41(1), 164–171.

Banos, O., Galvez, J. M., Damas, M., Pomares, H., & Rojas, I. (2014). Window size impact in human activity recognition. Sensors, 14(4), 6474 6499.

Ronao, C. A., & Cho, S. B. (2016). Human activity recognition with smartphone sensors using deep learning neural networks. Expert Systems with Applications, 59, 235–244.

Weiss, G. M., & Lockhart, J. W. (2012). The impact of personalization on smartphone-based activity recognition. AAAI Workshop on Activity Context Representation.